

# AI Meets Control Theory

From Classical Control to Intelligent Autonomous Systems

Living Technical Report — M1 — Phase 0 foundation

Generated by the project team (lead: puma; code: puma, toku; docs/research: lava, famo)

September 3, 2026

## Abstract

This report is regenerated at every project milestone. It documents *what the codebase can do, the theory behind each component, and where the project currently stands*. The project is an open-source laboratory for comparing classical control, optimal control, machine learning and reinforcement learning on dynamical systems, under the principle: *build every method from scratch, simulate it, visualise it, and compare it honestly*.

## Contents

|          |                                                                         |          |
|----------|-------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction and Scope</b>                                           | <b>1</b> |
| <b>2</b> | <b>Framework Architecture</b>                                           | <b>2</b> |
| 2.1      | Package layout . . . . .                                                | 2        |
| <b>3</b> | <b>Current Capabilities</b>                                             | <b>2</b> |
| <b>4</b> | <b>Theory: Modelling and Simulation</b>                                 | <b>2</b> |
| <b>5</b> | <b>Theory: Classical Control — PID</b>                                  | <b>4</b> |
| 5.1      | Worked example: stabilising an unstable plant (Experiment 03) . . . . . | 4        |
| <b>6</b> | <b>Theory: Modern Control — Pole Placement and LQR</b>                  | <b>5</b> |
| <b>7</b> | <b>Project Status and Roadmap</b>                                       | <b>5</b> |

## 1 Introduction and Scope

The central question of the project is: **when should we use classical control, when should we use AI, and when should we use both?** Rather than assuming a neural network or an RL agent is the answer, every problem is first attacked with PID, state feedback, LQR and MPC, and each method is measured against the others on identical conditions.

Every topic follows the same cycle:

Theory → Derivation → Implementation → Simulation  
→ Visualisation → Validation → Comparison → Experiment.

Fundamental algorithms are implemented from scratch in `numpy`; libraries (`scipy`, `python-control`) are used only to cross-check.

## 2 Framework Architecture

The reusable library is the Python package `aimct` (`src/aimct/`). Its design separates three concerns behind small interfaces so that any system, any controller and the simulator compose freely.

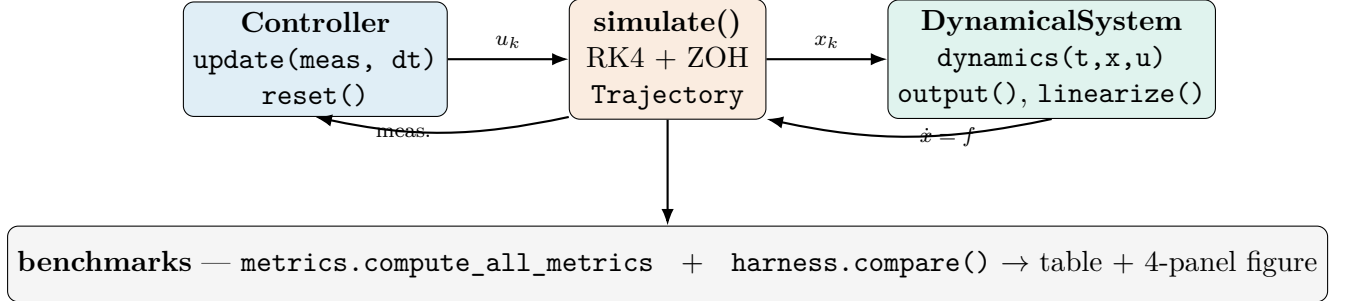


Figure 1: Core interfaces. The simulator drives a controller (`update(measurement, dt)`) and a system (`dynamics(t,x,u)`) through a fixed-step RK4 loop with zero-order hold, returning aligned time/state/input/output arrays. Optional hooks: `measurement_fn` (what the controller sees) and `input_disturbance` (additive plant-input disturbance).

### 2.1 Package layout

```

src/aimct/
  systems/      DynamicalSystem base + LinearSystem, MassSpringDamper,
                  Pendulum, CartPole (analytic Jacobians)
  simulate.py   rk4_step(), simulate() -> Trajectory(t, x, u, y)
  controllers/  Controller base + PID, StateFeedback, LQR
  benchmarks/   metrics.py (13 metrics), harness.py (compare())
  plot_style.py Okabe-Ito palette + canonical 4-panel comparison figure
  estimation/   ml/ rl/ (reserved for later phases)
  
```

## 3 Current Capabilities

## 4 Theory: Modelling and Simulation

A system is a first-order ODE  $\dot{x} = f(t, x, u)$  with output  $y = g(t, x, u)$ . Second-order mechanics are written in state form, e.g. the mass-spring-damper  $m\ddot{q} + c\dot{q} + kq = u$  becomes

$$\frac{d}{dt} \begin{bmatrix} q \\ \dot{q} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \begin{bmatrix} q \\ \dot{q} \end{bmatrix} + \begin{bmatrix} 0 \\ 1/m \end{bmatrix} u.$$

**Linearisation.** About an equilibrium  $(x_e, u_e)$ ,  $A = \partial f / \partial x|_e$ ,  $B = \partial f / \partial u|_e$ . The base class provides a central-difference Jacobian; each concrete system also supplies the closed-form  $A, B$  (the test suite asserts the two agree).

**Integration.** The simulator advances the state with classical fourth-order Runge–Kutta,

$$x_{k+1} = x_k + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4),$$

holding  $u$  constant across each step (zero-order hold), which matches how a discrete-time controller actually drives a plant.

| Component                       | What it provides                                                                                                                                                                                                                  | Tests |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| <code>systems</code>            | Continuous-time models with a common <code>dynamics(t,x,u)</code> interface, full-state or custom output, and numerical <i>and</i> analytic <code>linearize()</code> . Ships: linear SS, mass-spring-damper, pendulum, cart-pole. | ✓     |
| <code>simulate</code>           | Fixed-step RK4 with zero-order hold; <code>Trajectory</code> container; input saturation; measurement and disturbance hooks.                                                                                                      | ✓     |
| PID                             | From scratch: derivative-on-measurement with first-order filter, conditional-integration anti-windup, scalar or vector loops.                                                                                                     | ✓     |
| <code>StateFeedback</code>      | Pole placement via Ackermann's formula (SISO).                                                                                                                                                                                    | ✓     |
| LQR                             | Infinite-horizon regulator; CARE solved from scratch (Hamiltonian eigenvector method), cross-checked against <code>scipy.linalg.solve_continuous_are</code> .                                                                     | ✓     |
| <code>benchmarks.metrics</code> | 13 step-response / effort metrics: rise & settling time, overshoot, $e_{ss}$ , RMSE, IAE, ITAE, ISE, control energy, peak, slew, saturation duty.                                                                                 | ✓     |
| <code>benchmarks.harness</code> | <code>compare()</code> runs $N$ controllers on one system under identical conditions, emits the standard Markdown/CSV table and the 4-panel figure.                                                                               | ✓     |

Table 1: Implemented as of M1 — Phase 0 foundation. See `docs/roadmap.md` for what is next.

## 5 Theory: Classical Control — PID

The control law  $u(t) = K_p e(t) + K_i \int_0^t e \, d\tau + K_d \dot{e}(t)$ , with  $e = r - y$ , and the derivative acting on a filtered measurement to avoid derivative kick. Discrete form with step  $h$ :

$$I_k = I_{k-1} + e_k h, \quad D_k = D_{k-1} + \frac{h}{\tau_d + h} (D_k^{\text{raw}} - D_{k-1}), \quad u_k = K_p e_k + K_i I_k + K_d D_k.$$

Under saturation, *conditional integration* freezes  $I_k$  whenever the unsaturated command is outside the limits and the error would push it further out — preventing windup.

### 5.1 Worked example: stabilising an unstable plant (Experiment 03)

Plant with a right-half-plane pole,  $\ddot{y} - \omega_0^2 y = u + d$ ,  $\omega_0 = 2$ , unit step reference, step input disturbance  $d = 0.5$  at  $t = 5$  s, actuator limit  $|u| \leq 10$ .

- **P-only** cannot stabilise it: the closed loop is  $s^2 + (K_p - \omega_0^2)$  — poles on the imaginary axis, and once the actuator saturates on this unstable plant the response diverges.
- **PD** places both poles in the left half-plane, but the closed-loop DC gain from  $r$  is  $K_p / (K_p - \omega_0^2) = 20/16 = 1.25$ , so PD tracks the step with a permanent **25% offset**.
- **PID** removes the steady-state error for both the step and the disturbance.
- **Anti-windup** on the same gains cuts overshoot from 53%  $\rightarrow$  39% and IAE from 0.93  $\rightarrow$  0.80.

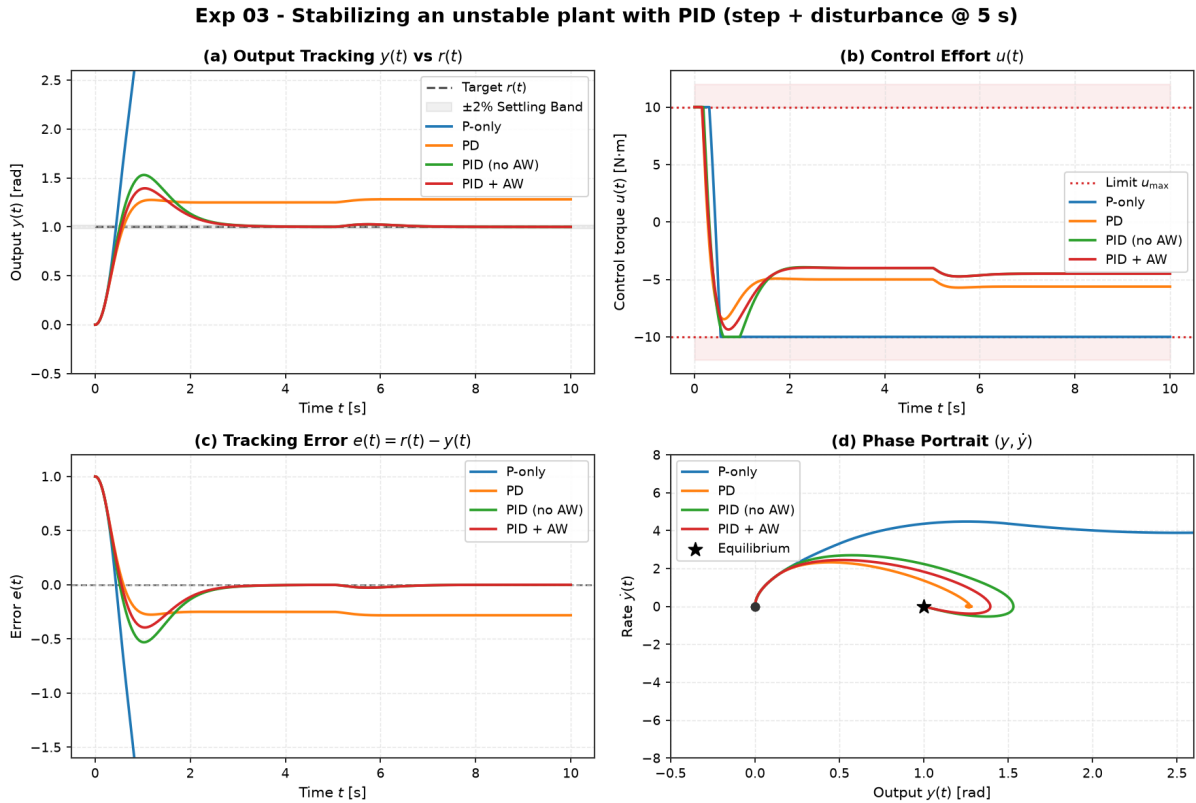


Figure 2: Experiment 03 — P / PD / PID / PID+anti-windup on the unstable second-order plant. Panels: (a) output tracking, (b) control effort with the actuator limit, (c) tracking error, (d) phase portrait. Regenerated from `experiments/03_pid_stabilizes_unstable/run.py`.

## 6 Theory: Modern Control — Pole Placement and LQR

For  $\dot{x} = Ax + Bu$  with  $(A, B)$  controllable, state feedback  $u = -Kx$  can place the closed-loop eigenvalues of  $A - BK$  anywhere. `StateFeedback` uses Ackermann's formula. `LQR` instead *chooses* the poles implicitly by minimising

$$J = \int_0^\infty (x^\top Q x + u^\top R u) dt, \quad u = -R^{-1} B^\top P x,$$

where  $P$  solves the continuous-time algebraic Riccati equation  $A^\top P + PA - PBR^{-1}B^\top P + Q = 0$ . We solve it from scratch via the stable invariant subspace of the Hamiltonian  $\begin{bmatrix} A & -BR^{-1}B^\top \\ -Q & -A^\top \end{bmatrix}$ , and verify the residual and the `scipy` solution agree.

## 7 Project Status and Roadmap

**Milestone M1 — Phase 0 foundation — done.** Library skeleton, four reference systems, RK4 simulator, PID / StateFeedback / LQR, metrics, comparison harness, Experiment 03, CI on Python 3.10–3.12, module 01–05 theory notes.

**Next.**

- Experiment 04: LQR vs pole placement vs PID on the cart–pole.
- Module 04–05 notes paired with the new controllers.
- Observers + Kalman filter (`estimation/`).
- From-scratch linear MPC with constraints (Phase 2).

**Reproduce.**

```
pip install -e ".[dev,xcheck]"
pytest -q
python experiments/03_pid_stabilizes_unstable/run.py
```